

PSReminderLite Help¹

<https://jdhitsolutions.github.io>

v1.1.1

¹<https://github.com/jdhitsolutions/PSReminderLite>

Contents

PSReminderLite	2
Introduction	2
Module Commands	3
Setup the Database	4
Module Preferences	4
Tags	5
Style	6
Exporting Preferences	6
Adding a Reminder	7
Getting Reminders	7
Modifying a Reminder	8
Archiving Reminders	8
Display Formatting	10
Database Management	12
Exporting the Database	14
Importing a Database	14
Importing Data	14
Removing Data	15
Migrating from MyTickle	15
Module Notes	16
Known Limitations	17
Related Projects	17
Roadmap	17
Credits	18

PSReminderLite



Introduction

This module is a port of the [MyTickle](#) module. The original module used an instance of SQLServer Express to provide a database of event reminders. This version uses SQLite to provide the same functionality. When you install the module from the PowerShell Gallery, it will also install the [MySqlite](#) module if it isn't already installed.

```
Install-Module PSReminderLite
```

OR

```
Install-PSResource PSReminderLite
```

The module requires a 64-bit PowerShell 7 platform. Windows on ARM64 is supported.

Note: *The module has not been tested extensively on non-windows platforms.*

Once the module is installed, you can run `Get-AboutPSReminder` to see the module information.

```
PS C:\> Get-AboutPSReminder
```

```
ModuleName      : PSReminderLite
Version          : {1.0.0, 1.1.0}
MySQLite         : 1.1.0
SQLiteVersion    : 3.42.0
PSVersion        : 7.5.2
Platform         : Win32NT
Host             : ConsoleHost
```

This may be helpful when filing an issue or asking for help.



Module Commands

All module commands should have full help with examples. The About help topic is a subset of this document.

Name	Alias	Synopsis
Add-PSReminder	<i>apsr, New-PSReminder</i>	Add a PSReminder to the database.
Export-PSReminderDatabase		Export the PSReminder database.
Export-PSReminderPreference		Export PSReminder variables.
Get-AboutPSReminder		Get module information.
Get-PSReminder	<i>gpsr</i>	Get one or more PSReminder entries.
Get-PSReminderDBInformation		Get information about the PSReminderEventDB database.
Get-PSReminderPreference		Get PSReminder preferences
Get-PSReminderTag	<i>gprr</i>	Get defined PSReminder tags
Import-FromTickleDatabase		Import data from a Tickle database
Import-PSReminderDatabase		Import PSReminder data from a JSON file.
Initialize-PSReminderDatabase		Initialize a new PSReminder database.
Move-PSReminder	<i>Archive-PSReminder</i>	Archive an expired PSReminder.
Open-PSReminderLiteHelp		Open a PDF help file.
Remove-PSReminder	<i>rpsr</i>	Delete a PSReminder from the database.
Set-PSReminder	<i>spsr</i>	Modify a PSReminder.

Note: Screenshots might vary slightly from the current version.



Setup the Database

This module uses SQLite to store event reminders. The first time you use the module, you will need to initialize the database. This will create a new SQLite database file in your user profile folder. You can specify a different location and name if you prefer but remember to update the \$PSReminderDB preference variable.

Initialize-PSReminderDatabase

The default database is \$HOME\PSReminder.db.

```
PS C:\> Get-Item $home\psreminder.db
```

Directory: C:\Users\Jeff

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a---	7/22/2024 1:38 PM	81920	psreminder.db

Note: If you uninstall the PSReminderLite module you will need to manually delete this file.

Module Preferences

The module exports several variables that are used to control the behavior of the module.

```
PS C:\> Get-Variable PSReminder*
```

PSReminderAlertStyle	
PSReminderArchiveTable	ArchivedEvent
PSReminderDB	C:\Users\Jeff\PSReminder.db
PSReminderDefaultDays	14
PSReminderExpiredStyle	
PSReminderTable	EventData
PSReminderTag	{[Event,],[Priority,],[Holiday,],...}

You should not modify the table and PSReminderDB variables after you initialize a database. If you want your database to use different table names or a different location, modify these variables **before** you initialize the database.

The PSReminderDefaultDays variable is used to determine how many days in the future to display reminders. The module default is 7 days. You can change this value to whatever you prefer.

Tags

This module allows you to tag reminders. The module exports a hashtable variable called `PSReminderTag`. This variable is used to define the tags you can use and an ANSI escape sequence which is used in the default reminder display.

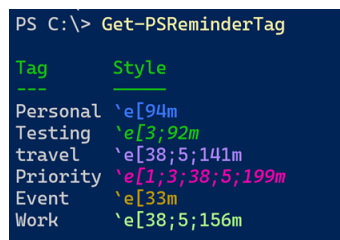
The module will define and use this variable by default.

```
$PSReminderTag = @{
    'Work'      = "`e[38;5;192m"
    'Personal'  = "`e[96m"
    'Priority'   = "`e[1;3;38;5;199m"
}
```

You can use native ANSI sequences or `$PSStyle` references. You can add as many tags as you would like.

```
$PSReminderTag.Add("Event", $PSStyle.Foreground.Yellow)
```

Run `Get-PSReminderTag` to see the current tags.

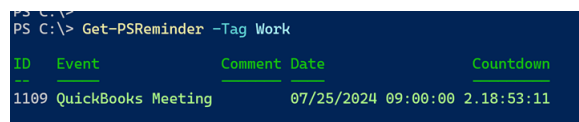


```
PS C:\> Get-PSReminderTag

Tag      Style
---
Personal `e[94m
Testing  `e[3;92m
travel   `e[38;5;141m
Priority  `e[1;3;38;5;199m
Event    `e[33m
Work     `e[38;5;156m
```

Figure 1: Get-PSReminderTag

Commands that have a tag-related parameter should have tab completion for defined tags. If the reminder has a defined tag, it will be displayed in the reminder list.



```
PS C:\> Get-PSReminder -Tag Work

ID  Event          Comment Date          Countdown
--  -
1109 QuickBooks Meeting 07/25/2024 09:00:00 2.18:53:11
```

Figure 2: Display reminders

You can specify multiple tags, but when it comes to formatting the default display, only the first tag will be used assuming no other conditions have been met.

There are no commands to add or remove tags. The `$PSReminderTag` variable is a hashtable that you can modify using PowerShell.

```
$PSReminderTag.Add("Private", "`e[38;5;133m")
```

Style

When you display reminders with `Get-PSReminder`, event due within 24 hours are style according to an alert style. Events due within 48 hours are styled with a warning style. Expired events are styled with an expired style. Beginning in version 1.1.0, you can now customize these styles.

```
$PSReminderAlertStyle = "`e[91m"  
$PSReminderWarningStyle = "`e[93m"  
$PSReminderExpiredStyle = "`e[9;38;5;163m"
```

You can see the values when you run `Get-PSReminderPreference`. Or if you save the object, you can invoke the `ShowStyle()` method.

```
PS C:\> $p = Get-PSReminderPreference  
PS C:\> $p.ShowStyle()  
  
Name                Style  
-----  
PSReminderAlertStyle  `e[91m  
PSReminderWarningStyle `e[93m  
PSReminderExpiredStyle `e[9;38;5;171m
```

Figure 3: ShowStyle

Note: *These settings have been modified from the module defaults*



Exporting Preferences

If you modify any of the PSReminder preference variables and want to make them persistent, it is recommended that you export them.

```
Export-PSReminderPreference
```

When you import the module, if the `$HOME\.psreminder.json` file exists, it will be imported into your session, defining the preference variables.

Note: *If you uninstall the PSReminderLite module you will need to manually delete this file.*

Adding a Reminder

To add a reminder, use the `Add-PSReminder` command. You must specify a date and time and a name for the event. You can optionally add a comment and/or tags.

```
PS C:\> Add-PSReminder -Date "9/1/2024 12:00PM" -Event "Labor Day" -PassThru
```

ID	Event	Comment	Date	Countdown
1107	Labor Day		9/1/2024 12:00 PM	40.22:21:55

This command has aliases of `New-PSReminder` and `apsr`.



Getting Reminders

The `Get-PSReminder` function has several self-explanatory parameters for displaying reminders from the database.

```
PS C:\> ( Get-Command Get-PSReminder -Syntax ) -replace "\\[\<common.*", ""  
Get-PSReminder [-Next <int>] [-DatabasePath <string>]
```

```
Get-PSReminder [-Id <int>] [-DatabasePath <string>]
```

```
Get-PSReminder [-EventName <string>] [-DatabasePath <string>]
```

```
Get-PSReminder [-All] [-DatabasePath <string>]
```

```
Get-PSReminder [-Expired] [-DatabasePath <string>]
```

```
Get-PSReminder [-Archived] [-DatabasePath <string>]
```

```
Get-PSReminder [-Year <int>] [-Month <int>] [-DatabasePath <string>]
```

```
Get-PSReminder [-Tag <string>] [-DatabasePath <string>]
```

The default is to display reminders for the next X number day as defined by the `$PSReminderDefaultDays` variable.



Modifying a Reminder

You can modify a reminder with the Set-PSReminder command. You must specify the ID of the reminder you want to modify. You can change the date, event name, comment, and tags.

```
PS C:\> Set-PSReminder -ID 1107 -Tags Personal
PS C:\> Set-PSReminder -id 1108 -Comment "online" -PassThru
```

ID	Event	Comment	Date	Countdown
1108	PSTweetChat	online	8/2/2024 13:00 PM	10.22:38:44



Archiving Reminders

The PSReminder and ArchivePSReminder objects are based on PowerShell class definitions, which have extended as well as hidden properties.

```
Class PSReminder {
    [String]$Event
    [DateTime]$Date
    [String]$Comment
    [int32]$ID
    [string[]]$Tags
    [boolean]$Expired = $False
    #add a hidden property that captures the database path
    hidden [string]$Source
    #add a hidden property to capture the computer name'
    hidden [string]$ComputerName = [System.Environment]::MachineName

    #constructor
    PSReminder([int32]$ID, [String]$Event, [DateTime]$Date,
    [String]$Comment, [String[]]$Tags) {
        $this.ID = $ID
        $this.Event = $Event
        $this.Date = $Date
        $this.Comment = $Comment
        $this.Tags = $Tags
        if ($Date -lt (Get-Date)) {
            $this.Expired = $True
        }
    }
}
} #close PSReminder class
```

```

Class ArchivePSReminder {
    [String]$Event
    [DateTime]$Date
    [String]$Comment
    [int32]$ID
    [string[]]$Tags
    [DateTime]$ArchivedDate
    #add a hidden property that captures the database path
    hidden [string]$Source
    #add a hidden property to capture the computer name'
    hidden [string]$ComputerName = [System.Environment]::MachineName

    ArchivePSReminder([int32]$ID,[String]$Event,[DateTime]$Date,
    [String]$Comment,[String[]]$Tags,
    [DateTime]$ArchivedDate) {
        $this.ID = $ID
        $this.Event = $Event
        $this.Date = $Date
        $this.Comment = $Comment
        $this.ArchivedDate = $ArchivedDate
        $this.Tags = $Tags
    }
} #close ArchivePSReminder class

Class PSReminderPreference {
    [string]$PSReminderDB = $global:PSReminderDB
    [string]$PSReminderDefaultDays = $global:PSReminderDefaultDays
    [string]$PSReminderTable = $global:PSReminderTable
    [string]$PSReminderArchiveTable = $global:PSReminderArchiveTable
    [hashtable]$PSReminderTag = $global:PSReminderTag

    [object]ShowTags () {
        $r = $this.PSReminderTag.GetEnumerator() | Foreach-Object {
            $stringValue = $_.Value.Replace("$([char]27)", '`e')
            [PSCustomObject]@{
                PSTypeName = 'PSReminderTag'
                Tag         = $_.Key
                Style        = '{0}{1}{2}' -f ($_.Value), $stringValue,
                $("`e[0m")
            }
        }
        return $r
    }
} #close PSReminderPreference class

```

```
Update-TypeData -TypeName PSReminder -DefaultDisplayPropertySet ID,Date,
Event,Comment -Force
Update-TypeData -TypeName PSReminder -MemberType AliasProperty
-MemberName Name -Value Event -Force
Update-TypeData -Type PSReminder -MemberType ScriptProperty
-MemberName Countdown -value {
    $ts = $this.Date - (Get-Date)
    if ($ts.TotalMinutes -lt 0) {
        $ts = New-TimeSpan -Minutes 0
    }
    $ts
}
```

By default, expired reminders are not displayed unless explicitly requested.

```
Get-PSReminder -Expired
```

These items remain in the primary event table. It is recommended that you periodically archive expired reminders. You can do this with the `Move-PSReminder` command. This command has an alias of `'Archive-PSReminder'`.

```
Get-PSReminder -Expired | Move-PSReminder
```

This is a manual process because you may want to review and adjust expired events before archiving them. If you want to automate the process, add the above command to your PowerShell profile script.

Note: *There is no module command to move an archived reminder back to the event table.*

You can use `Get-PSReminder` to view archived events.

```
PS C:\> Get-PSReminder -Archived | Select-Object -last 3
```

ID	Event	Comment	Date	ArchivedDate
1084	PSTweetChat		5/3/2024 1:00:00 PM	7/22/2024 2:05:01 PM
1101	Onramp 25 Planning		5/31/2024 2:00:00 PM	7/22/2024 2:05:01 PM
1085	PSTweetChat		6/7/2024 1:00:00 PM	7/23/2024 8:15:22 AM



Display Formatting

The default display using `Get-PSReminder` uses a custom formatting file. The default view will show expired items using a default ANSI escape sequence.

Items that will be due in 24 hours will be highlighted in red. Items that are

```
PS C:\> Get-PSReminder -Expired | Select -last 2
```

ID	Event	Comment	Date	Countdown
1098	BoA-Meeting	Slack-Huddle	07/16/2024 14:00:00	00:00:00
1073	Cmdlet-Working-Group		07/17/2024 12:00:00	00:00:00

Figure 4: Expired formatting

due in 48 hours will be highlighted in yellow. Otherwise, if the item is tagged and there is a definition in \$PSReminderTag, the ANSI escape sequence will be used.

```
PS C:\> Get-PSReminder -days 4
```

ID	Event	Comment	Date	Countdown
1114	Morning standup		07/23/2024 08:00:00	15:17:18
1102	Haircut		07/24/2024 08:30:00	1.15:47:18
1109	QuickBooks Meeting		07/25/2024 09:00:00	2.16:17:18

Figure 5: PSReminder display

In this example, the last item is tagged as `Work` and the ANSI escape sequence for `Work` is used.

There is also a custom table view called `Date` which will group reminders by custom property of `Month Year`.

```
PS C:\> Get-PSReminder | Format-Table -view date
PS C:\> Get-PSReminder -days 45 | Format-Table -view date
```

Month: Jul 2024

ID	Event	Comment	Date
1114	Morning standup		7/23/2024 8:00:00 AM
1102	Haircut		7/24/2024 8:30:00 AM
1109	QuickBooks Meeting		7/25/2024 9:00:00 AM

Month: Aug 2024

ID	Event	Comment	Date
1112	Alpha Meeting		8/1/2024 8:00:00 AM
1108	PSTweetChat	online	8/2/2024 1:00:00 PM
1075	Cmdlet Working Group		8/7/2024 12:00:00 PM
076	Cmdlet Working Group		8/21/2024 12:00:00 PM

Month: Sep 2024

ID	Event	Comment	Date
----	-------	---------	------

This view does not use tag highlighting.

Get-PSReminderPreference will display the current preference settings using a custom view.

```
PS C:\> Get-PSReminderPreference

PSReminderPreference Settings

PSReminderDB: C:\Users\Jeff\PSReminder.db

PSReminderDefaultDays PSReminderTable PSReminderArchiveTable
-----
14                      EventData          ArchivedEvent

PSReminderStyle

Name                      Style
PSReminderAlertStyle      `e[91m
PSReminderWarningStyle    `e[93m
PSReminderExpiredStyle    `e[9;38;5;171m

PSReminderTags

Tag                      Style
Event                   `e[38;5;87m
Priority                 `e[1;3;38;5;199m
Holiday                 `e[38;5;225m
Testing                 `e[3;92m
Personal                `e[94m
travel                  `e[38;5;141m
Family                  `e[38;5;156m
Work                    `e[38;5;192m
```

Figure 6: Get-PSReminderPreference

Database Management

You can use the Get-PSReminderDBInformation command to get a summary of the database tables and the number of records in each table.

```
PS C:\> Get-PSReminderDBInformation
```

Database: C:\Users\Jeff\PSReminder.db [80KB]

Age	Reminders	Expired	Archived
00.01:42:29	84	61	754

This is a rich object.

```
PS C:\> Get-PSReminderDBInformation | Select-Object *
```

```
Age           : 1.14:54:09.4753532
Name          : PSReminder.db
Path          : C:\Users\Jeff\PSReminder.db
PageSize      : 4096
PageCount     : 21
Reminders     : 21
Expired       : 23
Archived      : 807
Size          : 86016
Created       : 7/23/2024 3:59:03 PM
Modified      : 8/11/2024 7:20:31 PM
Author        : THINKX1-JH\Jeff
Comment       : Imported from a Tickle database
Encoding      : UTF-8
SQLiteVersion : 3.42.0
Date          : 8/13/2024 10:14:40 AM
Computername  : PROSPERO
```

Note: The Age property is a script property.

The object has a RefreshInfo() method. If you save the object to a variable, you can refresh the object with the current database information.

```
PS C:\> $db = Get-PSReminderDBInformation
```

```
PS C:\> $db.GetType().Name
```

```
PSReminderDBInfo
```

```
PS C:\> $db | Get-Member RefreshInfo
```

```
TypeName: PSReminderDBInfo
```

Name	MemberType	Definition
RefreshInfo	Method	PSReminderDBInfo RefreshInfo()

```
PS C:\> $db.RefreshInfo()
```

```
Database: C:\Users\Jeff\PSReminder.db [84KB]
```

Age	Reminders	Expired	Archived
01.15:24:26	21	23	807

Since the SQLite database is stored as a single file, there are limited management tasks compared to a full SQL Server instance. It is recommended that you backup the database file with your standard file backup process.

Exporting the Database

```
Export-PSReminderDatabase -Path c:\temp\jh-psreminder.json
```

You must specify a JSON file. This command doesn't export the database structure, only the contents of the database.

Importing a Database

If you exported a PSReminder database using `Export-PSReminderDatabase`, you can import it and restore the database using `Import-PSReminderDatabase`.

```
Import-PSReminderDatabase -Path c:\temp\jh-psreminder.json
```

The default behavior is to create a new database file using the `$PSReminderDB` preference variable. This file must not exist before running `Import-PSReminderDatabase`. The function will create the database file and import the data into the appropriate tables.



Importing Data

The parameters for `Add-PSReminder` accept pipeline input for new events. This makes it easy to import data from a CSV file.

```
PS C:\> Import-Csv C:\temp\reminder.csv | Add-PSReminder -Verbose -PassThru
VERBOSE: [14:45:36.8202189 BEGIN ] Starting Add-PSReminder
VERBOSE: [14:45:36.8203945 BEGIN ] Running under PowerShell version 7.4.3
VERBOSE: [14:45:36.8209222 PROCESS] Adding event 'Alpha Meeting'
VERBOSE: Performing the operation "Add-PSReminder" on target
"[2024-08-01 08:00:00] ".
VERBOSE: [14:45:36.8211739 PROCESS] INSERT INTO EventData (EventDate,EventName,
EventComment,Tags) VALUES ('2024-08-01 08:00:00','Alpha Meeting','', 'testing')
VERBOSE: [14:45:36.8591463 PROCESS] Adding event 'Alpha Meeting 2'
VERBOSE: Performing the operation "Add-PSReminder" on target
```



```
"[2024-08-15 08:00:00] ".
VERBOSE: [14:45:36.8597296 PROCESS] INSERT INTO EventData (EventDate,EventName,
EventComment,Tags) VALUES ('2024-08-15 08:00:00','Alpha Meeting 2','','testing')
VERBOSE: [14:45:36.8717859 END ] Ending Add-PSReminder
```

ID	Event	Comment	Date	Countdown
1110	Alpha Meeting		8/1/2024 08:00 AM	9.17:14:23
1111	Alpha Meeting 2		8/15/2024 08:00 AM	23.17:14:23

Removing Data

If you need to delete a reminder, use the `Remove-PSReminder` command. You must specify the ID of the reminder you want to delete, although you can take advantage of the pipeline.

```
PS C:\> Get-PSReminder -Tag Testing | Remove-PSReminder -Category Reminder -WhatIf
What if: Performing the operation "Remove-PSReminder" on target "Event ID 1110".
What if: Performing the operation "Remove-PSReminder" on target "Event ID 1111"
```

You might want to export items before deleting them:

```
PS C:\> Get-PSReminder -Archived | where tags -contains holiday |
ConvertTo-Json | Out-File d:\temp\archived-holidays.json
PS C:\> Get-PSReminder -Archived | where tags -contains holiday |
Remove-PSReminder -Category Archived
```

Note: *Removing an item may affect ID numbering which auto increments. This is not a major problem but be aware that your IDs may not be sequential if you remove newly added items.*

Migrating from MyTickle

If you are a user of the `MyTickle` module, you might want to migrate that data to a new `PSReminder` database. The process is quite simple. First, make sure your Tickle database is up to date. When you import the data, archived events will be imported into the archived table. But you are welcome to archive tickle events before you migrate.

When ready export the Tickle database.

```
Export-TickleDatabase -Path c:\temp\tickledb.xml
```

It is expected that if you are migrating data you do not have into an existing `PSReminder` database. If you do, you will need to **delete it**. If you don't, you

will get a warning when you try to import the data.

Now, import the data into a new PSReminder database

```
Import-FromTickleDatabase -Path c:\temp\tickledb.xml
```

That's it! Archived events will be added to the archive table. Everything else will be added to the event table.

Note: Any events with apostrophes in the event name will have the apostrophe stripped out.

You may want to tag the imported events. You can do this with the Set-PSReminder command.

```
Get-PSReminder | where name -match 'PSHSummit' |  
Set-PSReminder -Tags travel,event
```

It is recommended that you archive expired events.

```
Get-PSReminder -Expired | Move-PSReminder
```

Note: Be sure to tag expired events BEFORE you archive them.

At this point, you can begin using the PSReminder commands and uninstall the MyTickle module.



Module Notes

This module has a few design differences from other modules you may be familiar with. The module uses localized string data for messaging. This means that the module can be localized for different languages. If you would like to contribute a translation, I would welcome a pull request.

This module also uses customized Verbose messaging using a private function.

```
PS C:\Scripts\PSReminderLite> Get-PSReminderTag -Verbose  
VERBOSE: [10:24:25.3448625 BEGIN ] Get-PSReminderTag-> Starting module function Get-PSReminderTag  
VERBOSE: [10:24:25.3452454 BEGIN ] Get-PSReminderTag-> Running the command with PowerShell version 7.5.0-preview.3  
VERBOSE: [10:24:25.3455795 PROCESS] Get-PSReminderTag-> Getting PSReminder Tags from $PSReminderTag  
VERBOSE: [10:24:25.3467725 PROCESS] Get-PSReminderTag-> Getting defined tags from the database not in $PSReminderTag  
VERBOSE: [10:24:25.3511123 END ] Get-PSReminderTag-> Ending module function Get-PSReminderTag  
Tag Style  
-----  
Personal e[94m  
Testing e[3;92m  
travel e[38;5;141m  
Work e[38;5;156m  
Priority e[1;3;38;5;199m  
Holiday e[38;5;225m  
Event e[38;5;87m
```

Figure 7: custom verbose messaging

The command is still using the Verbose message stream, but the message text has been customized using ANSI escape sequences. If you save the output to a file, the escape sequences will be preserved.

Known Limitations

This module has had minimal testing on non-Windows platforms. If you find a problem, please post an Issue. The module does not have commands for the following activities, although some of these items could be scripted using `Invoke-mysqliteQuery`.

- The database is not password-protected, although it is on the wish list.
- There are no module commands to modify items from the archive table, however you can delete them.
- There are no module commands for modifying the database metadata table.

Related Projects

If find this project useful, you might also be interested in these other projects:

- [PSProjectStatus](#) - A PowerShell module for tracking project status and tasks.
- [PSWorkItem](#) - A PowerShell 7 module for managing work and personal tasks or to-do items.
- [PSCalendar](#) - A set of PowerShell commands for displaying calendars in the console.



Roadmap

There aren't many enhancements planned for this module. If there is something you would like to see or if you have questions or comments, please use the repository's Discussions section. For bugs and other problems, please file an issue.

Credits



Module icon by [Icons8](#)